

Spec-Driven Development + AI-Assisted Coding Playbook

When planning mode in Claude Code/Cursor is not enough

Slide 1: Who This Is For

If you are a:

- Dev working on large codebases
- Dev tackling a major feature
- Vibe-coder who wants to build complex stuff but struggles with structured prompts

This playbook is for you.

Learn to combine technical specs with AI agents so they stop rebuilding what exists, stay on track, and ship features in reviewable chunks—not 10K-line monster PRs.

Slide 2: The Problem

LLM-assisted coding is hard to master—unless you're doing bug fixes, minor tweaks, or toy demos.

Plan mode works for small stuff. Cursor/Claude Code plan one feature at a time. Describe what you want, they write code, you review. Done.

Plan mode breaks on large features. AI agents lose context mid-task, rebuild existing code, or drift off-spec halfway through.

If you're building something too big for plan mode, you need a technical spec.

Slide 3: What's Spec-Driven Development?

Spec-driven development = doing the hard work *before* code gets written:

- Research your codebase
- Understand what already exists
- Break the feature into reviewable chunks

Context is king.

You control what flows into the model's context window—and context rot is real. Performance drops as you shove in unstructured information.

Solution: Index your entire codebase. Build a searchable graph of your repo. Write specs with full awareness of what's already there.

Slide 4: Writing the Spec

With codebase context, let AI help you write the spec.

Start with a high-level vision:

"Let's build a specification for [feature]. Ask me clarifying questions first."

Then iterate:

1. Answer questions in detail
 2. Review the spec and contracts (types, API interfaces)
 3. Refine until solid
-

Slide 5: Generate Multi-Stage Plan

Once the spec is solid, break it into stages. Each stage = one PR-sized chunk.

- Scoped for incremental review and deployment

- QA tasks included per stage
- No 10K-line PRs—multiple focused PRs that actually get reviewed

The full feature, planned upfront, split into shippable pieces.

Slide 6: Generate Detailed Task List

For each stage, generate tasks with goals and acceptance criteria.

- Each task scoped for autonomous AI execution
- Review happens at stage end, via PR
- Clear acceptance criteria (what "done" looks like)

Stages = what gets shipped. Tasks = how it gets built.

Slide 7: Sharing Specs

Specs should be first-class artifacts—shareable, versioned, reviewable. They scale beyond one person.

Specs are the contract between humans, and between humans and AI agents.

You should be able to:

- Spec out a feature from terminal or IDE, in codebase context
- Upload to shared workspace with one action
- Give your team instant access with version history
- Share externally via a simple link

The spec becomes the source of truth—not a chat thread buried in Slack.

Slide 8: Pass It to Your AI Agent

If your spec contains:

- **Codebase research** – understands existing patterns and architecture
- **Implementation stages** – broken into reviewable PR-sized chunks
- **File-by-file instructions** – exact changes for each file in each stage

...pass it to your AI coding agent—Claude Code, Cursor, Codex, Antigravity—and watch it build the right thing.

Slide 9: Before/After Example

[Space for concrete example—e.g., LiteLLM case study or similar]

Slide 10: Shotgun

Looking for a tool that does all of this?

- Codebase indexing
- Spec generation
- Multi-stage planning
- Task breakdown
- Shareable specs

Check out Shotgun.

[See example spec generated by Shotgun](#)

Open source. BYOK.

shotgun.sh